Figure 1: The basic structure of RabbitMQ

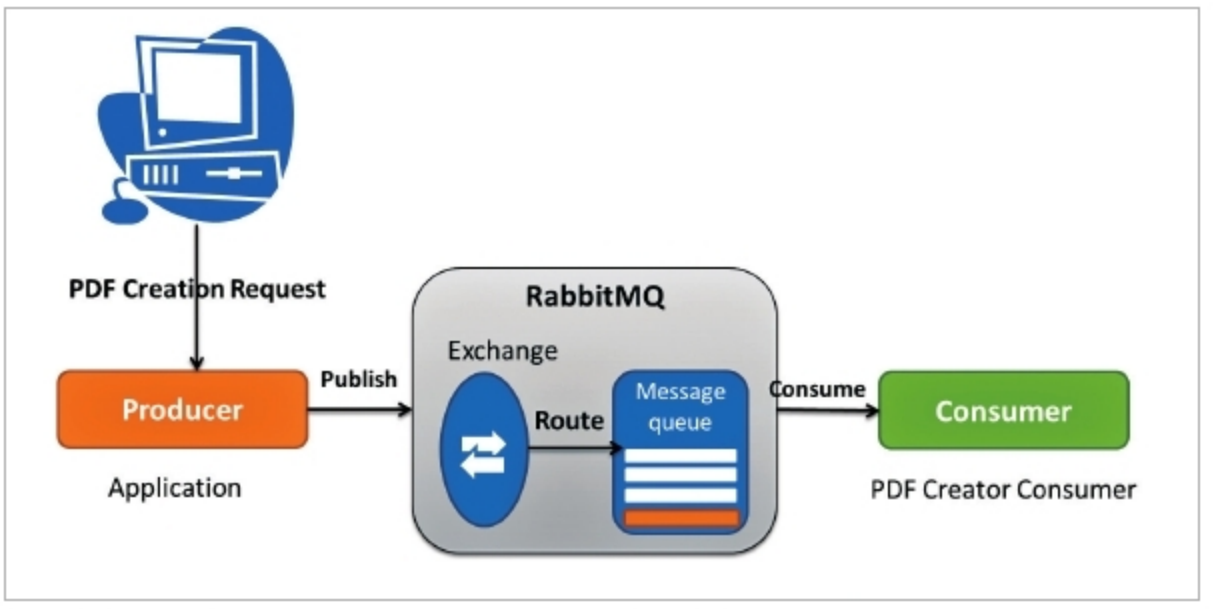

Figure 2: RabbitMQ message flow

Message queueing enables servers to perform tasks quickly. It balances the load between different workers when the message needs to be distributed to multiple recipients.

Once the producer puts in all the information, the consumer can retrieve the message from the queue and start generating the PDF at the same time as the producer is piling up the same queue with new messages. Here the consumer can be a server rather than a producer, or both can be at the same server. This task is independent of the languages used. So requests can be created in one language and handled in another. Both applications will communicate only through loosely coupled messages.

Producer messages are not directly published to queues. They are sent to an exchange, which decides the best route for them. The exchange routes the message to the proper consumer with the help of routing keys and bindings. A binding is nothing but a key between the queue and the exchange.

RabbitMQ features

From startups to large enterprises, RabbitMQ is the most widely used and deployed open source message broker.

Its features are listed below:

- Asynchronous messaging
- Routes messages using typical routing logic
- Provides delivery acknowledgement
- Programming language support
- Supports any programming language you can think of

Distributed deployment

Various RabbitMQ servers working on a local network can be clustered together to provide high availability and output for multiple zones because of the following reasons:

- RabbitMQ is easy to deploy in clouds
- Comes with pluggable authentication, and is also quick and easy to deploy in private or public clouds
- Has a very simple management UI, which makes monitoring and managing message queues a lot easier
- Quick integration
- Comes with various tools and plug-ins, which extend their functionalities and integrate with different systems. You can also develop your own plug-in and integrate it with RabbitMQ
- Supports messaging over multiple protocols
- Message tracing is possible with RabbitMQ
- If something goes wrong with

Table 1

Term	Meaning
Producer	An application that sends the messages
Consumer	An application that receives the messages
Queue	A buffer that saves messages
Message	A collection of data that needs to be sent from producer to consumer using RabbitMQ
Connection	A TCP connection acquired between the application and the RabbitMQ message broker
Channel	A virtual connection created inside a connection. All publishing and consuming of messages is done over a channel
Exchange	Retrieves messages from the producer and publishes them to an appropriate queue by using the rules defined on that exchange type
Binding	A link between an exchange and a queue
Routing key	The exchange uses this key to decide the routing of messages to queues. It can be called an address for the message
AMQP	Advanced Message Queuing Protocol – it is used by RabbitMQ for message routing
Users	Users accessing a RabbitMQ instance with given credentials. They may have different access permissions like read, write or configure the instance
Vhost	A virtual host that gathers applications using the same RabbitMQ instance